

03 动态规划入门

学习目标

- 能把题目中的“当前最优”改写成明确的 dp 状态。
- 能区分“必须以当前位置结尾”“考虑到当前位置为止”“当前容量/当前状态”的不同含义。
- 能完成考试风格代码填空，而不是只会写简单递推式。
- 能处理一维 DP、二维路径 DP、0/1 背包、双状态 DP、二进制状态 DP 的基础变式。

什么是动态规划

动态规划 (Dynamic Programming, 简称 DP) 是一种“把大问题拆成小问题，再用小问题答案推出大问题答案”的方法。它不是某一个固定公式，而是一套解题思路：

1. 先把原问题拆成若干个规模更小、结构相似的子问题。
2. 把已经算过的子问题答案保存下来，避免重复计算。
3. 按照明确的转移关系，从小规模一步步推出大规模。

适合用动态规划的题，通常有三个信号：

- 重叠子问题：同一个小问题会被反复遇到。
- 可递推结构：当前答案能由更小的答案推出。
- 无后效性：状态确定以后，后面的计算只需要这个状态值，不需要重新关心它是怎么来的。

可以把 DP 理解成一句话：**先定义清楚“我要记住什么”，再规定“怎么从已经记住的结果推出新结果”。**

入门案例：爬楼梯

楼梯有 n 级，每次可以走 1 级或 2 级，问走到第 n 级共有多少种走法。

先不要急着写代码，按 DP 的四个问题来分析：

1. 状态：dp[i] 表示走到第 i 级台阶的方法数。
2. 转移：要到第 i 级，最后一步只有两种可能：
 - 从第 $i-1$ 级走 1 级上来；
 - 从第 $i-2$ 级走 2 级上来。

所以 $dp[i] = dp[i-1] + dp[i-2]$ 。

3. 初始：站在地面还没上台阶，记作 $dp[0] = 1$ ；走到第 1 级只有一种方法，所以 $dp[1] = 1$ 。
4. 答案：最终读取 $dp[n]$ 。

以 $n = 5$ 为例：

i	含义	dp[i]
0	还没上楼梯，作为起点	1
1	走到第 1 级	1
2	从 1 级上来，或从地面跨 2 级	2
3	$dp[2] + dp[1]$	3
4	$dp[3] + dp[2]$	5
5	$dp[4] + dp[3]$	8

代码如下：

```
n = 5
dp = [0] * (n + 1)
dp[0] = 1
dp[1] = 1

for i in range(2, n + 1):
    dp[i] = dp[i - 1] + dp[i - 2]

print(dp[n])
```

输出为 8。这就是动态规划比普通递归更有效的地方：普通递归会反复计算“走到第 3 级有几种方法”“走到第 2 级有几种方法”，而 DP 把每个 $dp[i]$ 只算一次、存一次、用多次。

核心模型

动态规划不是把所有题都写成 $dp[i]=dp[i-1]+dp[i-2]$ 。真正要先问清楚四件事：

1. 状态： $dp[\dots]$ 表示哪个范围、哪个位置、哪个限制下的最优值或方案数。
2. 转移：当前状态最后一步来自哪里，或者当前对象选与不选。
3. 初始：最小规模、边界行列、空集合、容量为 0 时是什么。
4. 答案：最后读 $dp[-1]$ 、 $\max(dp)$ 、最后一行最小值，还是某个完整状态。

常见骨架：

初始化 `dp`

`for` 当前规模/当前物品/当前状态 `in` 范围:
枚举可能来源
用来源状态更新当前状态
答案 = 指定位置或指定范围的最优值

例题1：最大连续利润期

出处：改编自 [202506-舟山-高二-期末-13](#)。

某新能源公司采集若干天的成本和收入数据。每天利润为 `income[i]-cost[i]`，最大利润期指利润和最大的连续日期区间。

第一周成本数据为：

```
cost = [21, 22, 20, 28, 21, 27, 32]
income = [15, 20, 30, 22, 35, 30, 47]
```

先转化为每日利润：

```
gain = [-6, -2, 10, -6, 14, 3, 15]
```

若设 `dp[i]` 表示“必须以第 `i` 天结尾的最大连续利润和”，则第 `i` 天有两种处理：

- 接在前一段后面：`dp[i-1] + gain[i]`
- 从第 `i` 天重新开始：`gain[i]`

补全代码：

```
gain = [-6, -2, 10, -6, 14, 3, 15]
dp = [0] * len(gain)
dp[0] = gain[0]
for i in range(1, len(gain)):
    dp[i] = max(____①____, ____②____)
ans = max(dp)
print(dp, ans)
```

参考答案：

```
dp[i-1] + gain[i]
gain[i]
```

dp = [-6, -2, 10, 4, 18, 21, 36] , 最大利润和为 36 , 对应第 3 天到第 7 天, 共 5 天。

练习1：最大连续和的区间位置

来源 [202506-舟山-高二-期末-13](#)。

某新能源公司发展了高速公路服务区新能源充电站的运维业务，经营的利润由收入和成本相减所得。在实际经营过程中，发现成本和收入的变化都具有周期性规律，为了优化运维工作，数据分析人员采集了近一个月的每日成本和收入数据，设计算法找出当月最大利润期（最大利润期：利润和最大的连续日期区间）。

(1) 当月第一周的成本数据为 21, 22, 20, 28, 21, 27, 32；收入数据为 15, 20, 30, 22, 35, 30, 47。则这周最大利润期的天数是 ▲ 。

(2) 编写 Python 实现读取当月成本数据和收入数据，输出当月最大利润期的日期范围。程序运行结果如下：

```
当月成本数据: [38, 25, 21, 49, 21, 22, 29, 28, 15, 17, 29, 41, 49, 40, 35, 35,
45, 40, 19, 50, 46, 35, 21, 21, 15, 20, 33, 28, 39, 39]
当月收入数据: [22, 24, 27, 41, 20, 40, 21, 44, 35, 40, 31, 21, 32, 24, 21, 37,
28, 24, 32, 21, 28, 32, 31, 23, 34, 29, 36, 31, 29, 41]
最大利润期: 位置: 第6天到第11天
```

请在划线处填入合适的代码。

```
# 读取当月成本数据存储到列表 cost 中，收入数据存储到列表 income 中，代码略
n = len(cost)
s = [0] * n
s[0] = income[0] - cost[0]
for i in range(1, n):
    s[i] = ①

max_sum = s[0]
start = 0
end = 0
for j in range(n):
    if s[j] > max_sum:
        max_sum = s[j]
        ②
        end = j
    for i in range(j):
        current_sum = s[j] - s[i]
        if current_sum > max_sum:
            ③
            start = i + 1
```

```
        end = j

print("最大利润期: ")
print("位置: 第 {} 天到第 {} 天".format(start + 1, end + 1))
```

参考答案:

(1) 5

(2) ① `s[i-1] + income[i] - cost[i]`; ② `start = 0`; ③ `max_sum = current_sum`

解析:

第(2)问原程序中的 `s[i]` 是前缀利润和:

```
s[i] = 第1天到第 i+1 天的利润总和
```

所以 `s[i] = s[i-1] + income[i] - cost[i]`。当区间从第 1 天开始时, 利润和直接是 `s[j]`, 此时起点应为 `start = 0`; 当区间从第 `i+2` 天开始、到第 `j+1` 天结束时, 区间利润和是 `s[j] - s[i]`。如果这个值更大, 就用 `max_sum = current_sum` 更新最大利润, 并记录 `start = i + 1`、`end = j`。

例题2: 不相邻选择

有一排任务奖励:

```
a = [6, 1, 2, 10, 4, 7]
```

不能选择相邻任务, 求最大总奖励。设 `dp[i]` 表示“以 `0..i` 这些任务为结尾时, 能取得的最大总奖励”。

补全代码:

```
a = [6, 1, 2, 10, 4, 7]
dp = [0] * len(a)
dp[0] = a[0]
dp[1] = _____
for i in range(2, len(a)):
    dp[i] = max(_____①_____, _____②_____)
print(dp[-1])
```

参考答案:

```
max(a[0], a[1])
dp[i-1]
dp[i-2] + a[i]
```

dp = [6, 6, 8, 16, 16, 23] , 最大总奖励为 23 , 可选 6、10、7。

练习2：状态压缩的不相邻选择

不相邻选择可以不用完整 dp 数组，只保留前两个状态。补全代码。

```
a = [4, 10, 3, 1, 5, 9]
pre2 = a[0]
pre1 = max(a[0], a[1])
for i in range(2, len(a)):
    cur = max(-----①-----, -----②-----)
    pre2 = -----③-----
    pre1 = cur
print(pre1)
```

参考答案：

```
pre1
pre2 + a[i]
pre1
```

输出 20 , 可选 10、1、9。

例题3：数字三角形最小路径和

出处：选自 [202406-宁波九校-高二-期末-13](#)。

给定一个数字三角形，找出自顶向下的最小路径和。每一步只能移动到下一行中相邻的结点上。相邻结点指的是下标与上一层结点下标相同或者等于上一层结点下标 +1 的两个结点。

```
a = [[2], [3, 4], [6, 5, 7], [4, 1, 8, 3]]
```

设 $f[i][j]$ 表示到达第 i 行第 j 个位置的最小路径和。

(1) 若将第四行第二列的数字 1 改为 7，则新的最小路径和为 _ _ _ _。

(2) 计算最小路径和的程序如下，请补全代码。

```

a = [[2], [3, 4], [6, 5, 7], [4, 1, 8, 3]]
n = len(a)
f = [[0] * n for i in range(n)]
f[0][0] = a[0][0]
for i in range(1, n):
    f[i][0] = f[i-1][0] + a[i][0]
    for j in range(1, i):
        if _____①_____:
            f[i][j] = f[i-1][j-1] + a[i][j]
        else:
            f[i][j] = f[i-1][j] + a[i][j]
    f[i][i] = _____②_____
    _____③_____
for i in range(1, n):
    if f[n-1][i] < ans:
        ans = f[n-1][i]
print("最小路径和为: ", ans)

```

参考答案：

(1) 15。

(2)

```

f[i-1][j-1] < f[i-1][j]
f[i-1][i-1] + a[i][i]
ans = f[n-1][0]

```

原三角形最小路径和为 11，路径为 2 + 3 + 5 + 1。

例题4：0/1 背包与任务分配

出处：改编自 [202512-强基联盟-高三-月考-15](#)。

在人工智能与高性能计算融合的场景中，需将若干计算任务分配到 n 个计算节点上。每个计算节点的算力上限为 m （单位：TFLOPS，万亿次浮点运算每秒），每个计算任务的算力需求不超过 m ，且所有任务的总算力需求恰好为 $n \times m$ ，任务不可拆分，为最大化资源利用率，需要让每个节点满负荷运行，需确定各节点的任务分配方案。

例如，有 8 个计算任务，各任务的应用场景及算力需求如第 15 题图 a 所示，现需将任务分配到 4 个计算节点，每个节点的算力上限为 1200 TFLOPS。计算节点的任务分配方案可能有多种，其中一种可行方案如第 15 题图 b 所示：

任务编号	任务类型	算力需求 (TFLOPS)
1	AI 大模型训练	1200
2	全球气候模拟	800
3	自动驾驶城市路况仿真	700
4	基因组批量测序分析	400
5	金融高频交易策略回测	200
6	电影特效复杂场景渲染	600
7	大语言模型预训练	500
8	卫星遥感图像 AI 解译	400

第 15 题图 a

计算节点	分配的任务编号
节点 1	1
节点 2	2、4
节点 3	3、7
节点 4	5、6、8

第 15 题图 b

请编写 Python 程序，实现计算节点任务分配方案。

- (1) 将上述 8 个计算任务分配到 3 个计算节点，每个计算节点的算力上限为 1600 TFLOPS。计算节点分配的任务编号情况为 `[[1,4], _____]`。
- (2) 定义如下 `sorttasks(data)` 函数，函数 `data` 的每个元素由“任务编号”、“任务类型”、“算力需求”3 个数据项组成。函数的功能是根据各任务算力需求降序排序。请在划线处填入合适代码。

```
def sorttasks(data):
    task = [[] for i in range(len(data))]
    x = 0
    for i in range(len(data)):
        t = data[i][2]
        j = x - 1
        while _____:
            task[j + 1] = task[j]
            j -= 1
        task[j + 1] = [data[i][0], t]
        x += 1
    return task
```

(3) 实现节点算力分配功能的部分 Python 程序如下，程序中用到的列表方法函数如下表所示，请在程序中划线处填入合适的代码。

函数与方法	功能
<code>x = lst.pop(0)</code>	将列表 <code>lst</code> 第一位元素赋值给 <code>x</code> ，并将其从 <code>lst</code> 中删除


```

def findflops():
    global num          # num 为全局变量
    a = [0] * (m + 1)
    lnk = [-1] * (m + 1)
    for i in range(cnt):
        if not flag[i]:
            t = tasks[i][1]
            for j in range(m, t - 1, -1):
                if a[j] < a[j - t] + t:
                    a[j] = a[j - t] + t
                    lnk[j] = i
    if a[m] == m:
        p = m
        while _____①_____:
            j = lnk[p]
            lst[num].append(tasks[j][0])
            q.append([num, j])
            flag[j] = True
            p -= tasks[j][1]
        num = (num + 1) % n
        return True
    return False

```

...

读取节点总数存入变量 `n`，单个计算节点的算力上限（TFLOPS）存入变量 `m`，
读取计算任务存入列表 `data` 中，代码略

...

```

tasks = sorttasks(data)
cnt = len(tasks)
num = 0
flag = [False] * cnt
lst = []
for i in range(n):
    lst.append([])
q = []          # 记录节点分配的任务编号情况
while len(q) != cnt:
    _____②_____
    if not found and q != []:
        tmp = q.pop(0)
        lst[tmp[0]] = []
    _____③_____

```

输出各计算节点分配的任务编号，代码略

参考答案：

```

(1) `[2,5,6],[3,7,8]`;
(2) `j >= 0 and t > task[j][1]`;
(3) ① `lnk[p] != -1`;
② `found = findflops()`;
③ `flag[tmp[1]] = False`

```

关键点：j 必须倒序枚举，保证每个任务最多使用一次；lnk[p] 用于从容量 m 反向找出本轮选了哪些任务。

其中 findflops() 用 0/1 背包思想，在尚未分配的任务中找出一组总算力恰好为 m 的任务。

练习4：0/1 背包代码填空

背包容量 $W=7$ ，物品 [重量, 价值] 如下：

```
items = [[2, 3], [3, 4], [4, 8], [5, 9]]
```

每个物品最多选一次。补全代码。

```

W = 7
items = [[2, 3], [3, 4], [4, 8], [5, 9]]
dp = [0] * (W + 1)
for w, v in items:
    for j in range(____①____, ____②____, ____③____):
        dp[j] = max(dp[j], ____④____)
print(dp[W])

```

参考答案：

```

W
w - 1
-1
dp[j-w] + v

```

输出 12。容量 7 时可选重量 2+5 或 3+4 的组合。

例题5：双状态 DP

某学生连续训练，每天可以“训练”或“休息”，但不能连续休息两天。第 i 天训练收益为 $a[i]$ ，求最大收益。

```
a = [4, 2, 7, 1, 6]
```

定义两个状态：

- `dp[i][0]`：第 `i` 天休息时，前 `0..i` 天能得到的最大收益。
- `dp[i][1]`：第 `i` 天训练时，前 `0..i` 天能得到的最大收益。

补全转移：

```
a = [4, 2, 7, 1, 6]
dp = [[0, 0] for i in range(len(a))]
dp[0][0] = 0
dp[0][1] = a[0]
for i in range(1, len(a)):
    dp[i][0] = -----①-----
    dp[i][1] = max(-----②-----, -----③-----) + a[i]
print(max(dp[-1]))
```

参考答案：

```
dp[i-1][1]
dp[i-1][0]
dp[i-1][1]
```

输出 20。限制“不连续休息”只影响休息状态：今天休息，则昨天必须训练。

例题6：二进制状态 DP

有 3 项能力需要补齐，分别用二进制位表示：

第0位：算法
第1位：数据结构
第2位：程序阅读

若一个训练包能覆盖若干能力，就用一个二进制状态表示。例如 `011` 表示覆盖“算法”和“数据结构”。

现有训练包：

```
packs = [
    [001, 3],
    [010, 4],
    [100, 5],
    [011, 6],
    [110, 7],
```

```
[101, 8]  
]
```

每个元素为 [覆盖能力, 时间]。求覆盖全部 3 项能力的最少时间。设 `dp[mask]` 表示已经覆盖能力集合 `mask` 时的最少时间。

补全代码：

```
def bit_or(a, b):  
    # 1. 转为 3 位的二进制字符串（例如：3 -> '011'）  
    存入str_a、str_b 中，代码略  
  
    res_str = ""  
    for i in range(3):  
        if _____:  
            res_str += "1"  
        else:  
            res_str += "0"  
  
    # 3. 将二进制字符串转回十进制数  
    return int(res_str, 2)  
  
INF = 10**9  
dp = [INF] * 8  
dp[0] = 0  
for cover, cost in packs:  
    for mask in range(8):  
        new_mask = _____①_____  
        if dp[new_mask] > _____②_____:  
            dp[new_mask] = _____③_____  
print(dp[7])
```

参考答案：

```
str_a[i] == "1" or str_b[i] == "1"  
bit_or(mask, cover)  
dp[mask] + cost  
dp[mask] + cost
```

输出 10，可选 0b001 和 0b110 两个训练包。

状态转移：当我们有一个现有的能力集合 `mask`，并且选择了一个新的训练包 (`cover, cost`) 时：

- 新的能力集合为当前集合与新训练包覆盖能力的按位或（OR）结果：`new_mask = mask | cover`
- 达到 `new_mask` 的新花费时间为：`dp[mask] + cost`

课后作业

作业1：老鼠试药与二进制状态

来源：[202412-强基联盟-高二-月考-17](#)。

老鼠试药：在医学研究中，常常需要进行动物实验来测试药物的安全性和有效性。假设有 n 个药瓶混合在一起，编号为 $0 \sim n-1$ ，其中有 1 瓶是毒药，另 $n-1$ 瓶是纯净水。已知老鼠服用毒药后会在 1 小时内死亡，服用纯净水对生命没有影响。我们需要使用尽可能少的老鼠来喂服这些药瓶中的液体，以便快速确定毒药瓶的编号。

如果使用二进制对药瓶编号，则只需要 $\text{ceil}(\log_2 n)$ 只老鼠即可快速找出药瓶。假设 $n=100$ ，则对药瓶用二进制编号如下表所示。从表中可知，一共需要 7 只老鼠试药，每只老鼠喂服二进制编码中纵向为 1 的药瓶，然后观察老鼠的生命状态。例如：1 小时后编号为 1,2,3,5,6 的老鼠死亡，即可知道二进制编码 $1 \times 2^0 + 1 \times 2^1 + 1 \times 2^2 + 1 \times 2^4 + 1 \times 2^5 = 55$ ，即编号为 55 的药瓶是毒药。

药瓶	第 6 位	第 5 位	第 4 位	第 3 位	第 2 位	第 1 位	第 0 位
0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	1
2	0	0	0	0	0	1	0
3	0	0	0	0	0	1	1
...							
55	0	1	1	0	1	1	1
56	0	1	1	1	0	0	0
...							

98	1	1	0	0	0	1	0
99	1	1	0	0	0	1	1

老鼠编号	鼠 7	鼠 6	鼠 5	鼠 4	鼠 3	鼠 2	鼠 1
喂服药瓶 编号	..., 98, 99	..., 98, 99	..., 55, 56 ...	..., 55, 56 ...	..., 56, ...	2, 3, ... 55, ... 98, 99	1, 3, ..., 55, ...99

编写 Python 程序模拟老鼠试药试验，程序运行界面如下图所示，请回答以下问题：

```
>>> %Run mouse.py
请输入药瓶数量: 100
最少需要7只老鼠试药
观察到以下老鼠壮烈牺牲: 12356
老鼠试药结果: 毒药瓶编号为 55
```

<code>ceil()</code>	向上取整数
<code>log(n, 2)</code>	即数学表达式 $\log_2 n$

(1) 假设 $n=100$ ，喂服液体没有差错，1 小时后没有发现老鼠死亡，此时可以推测毒药瓶的编号是 ▲。（单选，填字母：A. 0 / B. 1 / C. 100 / D. 不可能）

(2) 编写十进制数 x 转 k 位二进制数函数 `dtob()`，请补充程序中的代码。

```
def dtob(x, k):
    s = ''
    for i in ①:
        r, x = x % 2, x // 2
        s = str(r) + s
    return s
```

(3) 编写函数，输入药瓶数量，返回最少需要试药老鼠的数量，请将划线处代码补充完整。

```
import math

def mouse(n):
    m = math.log(n, 2) # 函数 log(n, 2) 即数学表达式  $\log_2 n$ 
    if m != int(m):
        ②
    else:
        m = int(m)
    return m
```

(4) 模拟老鼠试药：程序首先随机预置毒药瓶，根据二进制编码统计老鼠服药方案；然后按服药方案计算并输出老鼠的牺牲情况；最后，根据老鼠牺牲数据计算试药结果。请将划线处代码补充完整。

```
import random

n = int(input("请输入药瓶数量: "))
```

```

print(f"最少需要{mouse(n)}只老鼠试药")
poison = random.randint(0, n-1) # 随机预置毒药瓶编号
③
dic = {}
for i in range(n):
    temp = ④
    for j in range(m):
        if temp[j] == "1":
            if j not in dic:
                ⑤
            else:
                dic[j].append(i)

print("观察到以下老鼠壮烈牺牲：", end="")
d = 0
for i in dic:
    if ⑥:
        print(m - i, end=" ")
        d += 2 ** (m - i - 1)

print(f"\n老鼠试药结果：毒药瓶编号为{d}")

```

参考答案：

(1) A

(2) ① range(k)

(3) ② $m = \text{int}(m) + 1$ 或 $m = \text{math.ceil}(m)$

(4) ③ $m = \text{mouse}(n)$; ④ $\text{dtob}(i, m)$; ⑤ $\text{dic}[j] = [i]$; ⑥ $\text{poison in dic}[i]$

解析：

这题的关键不是普通十进制计算，而是把一个药瓶编号看成一个二进制状态。每只老鼠对应一位：这一位为 1，说明该老鼠会服用这个药瓶；最后死亡老鼠的编号组合起来，就能还原毒药瓶的二进制编号。

$\text{mouse}(n)$ 求的是至少需要多少个二进制位才能表示 $0 \sim n-1$ 这些编号，因此需要 $\text{ceil}(\log_2 n)$ 。如果没有老鼠死亡，说明毒药瓶编号的每一位都是 0，所以毒药瓶编号只能是 0。

在程序中， $\text{dic}[j]$ 记录“第 j 个二进制位为 1 的所有药瓶”。若 $\text{poison in dic}[i]$ ，说明第 i 位对应的老鼠会死亡，于是把这一位对应的权值 $2 ** (m - i - 1)$ 加到结果 d 中。

作业2：手机网格扫描

来源: [202509-嘉兴-高三-月考-15](#)。

手机屏幕上图标排列采用网格化布局, 图标严格对齐网格系统。设手机屏幕网格区域大小为 $W \times H$ (W 和 H 分别表示屏幕宽度与高度, 即横向与纵向网格数量), 手机图标尺寸大小有 n 种, 分别为 $w_0 \times h_0$ 、 $w_1 \times h_1$ 、 $\dots$ 、 $w_{n-1} \times h_{n-1}$ (w 和 h 表示图标宽度与高度), 当手机屏幕大小为 4×2 , 手机图标的大小为 1×1 、 2×2 、 4×2 , 要将图标填满整个屏幕的网格区域, 共 6 种放置方案, 如下图所示。



求解放置方案种数的算法规则如下:

扫描规则: 从屏幕左上角开始, 按从左到右、从上到下的顺序扫描每个网格位置;

放置规则: 若当前扫描位置未被占用(可放置图标), 则按图标索引顺序依次尝试放置所有规格的图标; 若可放置, 则标记该图标占用的所有网格为已占用, 继续扫描下一个未占用位置并重复放置流程;

回退规则: 当所有网格位置均被占用(屏幕填满)时, 记为 1 种有效方案; 若当前位置无法放置任何规格的图标, 则回退至上一步, 取消上一个图标的占用标记, 尝试其他可放置的图标规格。

(1) $W \times H$ 大小的屏幕使用二维数组(嵌套列表)存储。二维数组元素值 0 表示未占用, 1 表示已占用。例如: 二维数组 $[[1, 0, 0], [1, 0, 0]]$ 表示的屏幕, 有 2 个位置已被占用, 若在该屏幕中放入 1 个 2×1 大小的图标, 则该二维数组的值为 ▲。

(2) 函数 `find_pos` 用于查找能够放置图标的位置, 若找到则返回行列位置 `[row, col]`, 否则表示屏幕已放满, 返回 "满"。Python 程序如下, 请在划线处填入合适代码。

```
def find_pos(scr):  
    for row in range(H):  
        for col in range(W):  
            if ▲:  
                return [row, col]  
    return "满"
```

(3) 函数 `can_place` 用于判断屏幕中是否能放置图标。其中 `scr` 为表示屏幕状态的二维数组; `pos` 为要放置图标的起始位置, 数据结构为 `[row, col]`; `icon` 为图标大小, 数据结构为 `[w, h]`。如下程序加框处代码存在错误, 应修改为 ▲。

```
def can_place(scr, pos, icon):  
    row, col = pos[0], pos[1]  
    iw, ih = icon[0], icon[1]  
    if row + iw > H and col + ih > W: # 加框处  
        return False
```



```

for w in range(iw):
    for h in range(ih):
        if scr[row + h][col + w] == 1:
            return False
return True

```

(4) 统计屏幕放置图标方案种数的 Python 程序如下，请在划线处填入合适的代码。

```

W, H = 7, 5          # 手机屏幕的宽和高
ICONS = [[1, 1], [2, 2], [4, 2], [4, 4]]
scr = [[0] * W for i in range(H)]
pos = [0, 0]
# data 链表的每个节点的格式如下：
# [屏幕状态， 放置图标的位置， 尝试放置图标的索引， 方案种数， 链表指针]
data = [[scr, pos, 0, 0, -1]]
head = 0
while not (data[head][2] == len(ICONS) and data[head][4] == -1):
    if data[head][2] == len(ICONS):      # 当前节点处理完，回退
        next = data[head][4]
        data[next][3] += data[head][3]
        ①
    else:
        scr, pos, icon_idx = data[head][0], data[head][1], data[head][2]
        data[head][2] += 1
        if can_place(scr, pos, ICONS[icon_idx]):
            # 保存当前屏幕状态到 new_scr
            new_scr = [[0] * W for i in range(H)]
            for row in range(H):
                for col in range(W):
                    new_scr[row][col] = scr[row][col]
            # 标记该图标占用的所有网格为已占用
            iw, ih = ICONS[icon_idx][0], ICONS[icon_idx][1]
            for w in range(iw):
                for h in range(ih):
                    new_scr[pos[0] + h][pos[1] + w] = 1
            pos = find_pos(new_scr)
            if pos == "满":      # 全部放满
                ②
            else:      # 未全部放满
                new_head = [new_scr, pos, 0, 0, head]
                data.append(new_head)
                head = len(data) - 1
print("放置方案种数: ", data[0][3])

```

参考答案：

```
(1) [[1,1,1],[1,0,0]]

(2) scr[row][col] == 0

(3) row + ih > H or col + iw > W

(4) ① head = next (或 head = data[head][4]) ; ② data[head][3] += 1
```

解析

- (1) 2×1 图标 (宽 2 格、高 1 格) 放在 `[[1,0,0],[1,0,0]]` 中的右上角空闲区域 (第 0 行第 1-2 列) , 占用后数组变为 `[[1,1,1],[1,0,0]]` 。
- (2) 扫描查找第一个未被占用的位置 (值为 0) , 即 `scr[row][col] == 0` 。
- (3) 加框处原代码 `row + iw > H and col + ih > W` 有两个错误: ① 行列判断混淆 (iw 是宽度对应 col, ih 是高度对应 row) ; ② 应为 `or` 而非 `and` ——只要任一方向越界就不能放置。正确代码: `row + ih > H or col + iw > W` 。
- (4) 该程序使用链表结构实现回溯 (DFS) :
- ① 回退时, 将 head 指针移回上一节点: `head = next` 。
 - ② 全部放满时, 当前节点方案种数加 1: `data[head][3] += 1` 。最终 `data[0][3]` 汇总所有方案种数。

错题复盘栏

错题	错误状态	正确状态/转移

教师观察记录

观察点	表现
是否能独立定义 dp 状态	
是否能区分“以 i 结尾”和“考虑到 i”	
是否能处理二维边界	
是否理解 0/1 背包倒序	
是否能读懂二进制状态合并	